

***Facultad
de
Ciencias***

**DESARROLLO DE UNA APLICACIÓN
MÓVIL PARA LA PLANIFICACIÓN DE
RUTINAS DE EJERCICIO Y DIETAS**

**(Development of a mobile application for
planning exercise routines and diets)**

**Trabajo de Fin de Grado
para acceder al**

GRADO EN INGENIERÍA INFORMÁTICA

Autor: Alejandro Arana Gómez

Directora: Patricia López Martínez

Junio – 2026

ÍNDICE DE CONTENIDO

ÍNDICE DE CONTENIDO	i
ÍNDICE DE TABLAS	iii
ÍNDICE DE FIGURAS	iv
RESUMEN	v
ABSTRACT	vi
1. INTRODUCCIÓN	1
1.1 Motivación.....	1
1.2 Objetivos.....	1
1.3 Organización de la memoria	2
2. TECNOLOGÍAS Y HERRAMIENTAS.....	3
2.1 Entornos de desarrollo.....	3
2.2 Lenguajes y frameworks	3
2.3 Diseño y Pruebas	4
2.4 Comunicación y gestión de datos	4
2.5 Gestión del proyecto	4
3. ANÁLISIS DE REQUISITOS	5
3.1 Requisitos funcionales	5
3.2 Requisitos no funcionales	8
4. ARQUITECTURA Y DISEÑO	9
4.1 Arquitectura del sistema	9
4.2 Capa de presentación.....	9
4.3 Capa de negocio.....	10
4.4 Capa de datos	19
4.5 Modelo de dominio	16
4.5.2 Datos del usuario	17
4.5.3 Datos de las dietas.....	17
4.5.4 Datos de las rutinas.....	19
5. IMPLEMENTACIÓN	21
5.1 Implementación del backend	21
5.1.1 Capa de controladores	21
5.1.2 Capa de Servicios	22
5.1.3 Capa de Repositorios	22
5.2 Implementación del frontend.....	23
5.2.1 Capa Model.....	23
5.2.2 Capa View	24
5.2.3 Capa ViewModel	26
6. PRUEBAS	27
6.1 Pruebas del backend	27
6.1.1 Pruebas unitarias	27
6.1.2 Pruebas de integración	27
6.2 Pruebas del frontend.....	29
7. CONCLUSIONES Y TRABAJOS FUTUROS	32

7.1	Conclusiones	32
7.2	Trabajos Futuros	32
	Bibliografía	34

ÍNDICE DE TABLAS

Tabla 1 - Requisitos generales del sistema.....	5
Tabla 2 - Requisitos sobre dietas	6
Tabla 3 - Requisitos sobre rutinas de gimnasio.....	7
Tabla 4 - Requisitos No Funcionales.....	8
Tabla 5 - EndPoints Usuarios.....	11
Tabla 6 - EndPoints Alimentos Globales	11
Tabla 7 - EndPoints Alimentos del Usuario	11
Tabla 8 - EndPoints Dietas.....	12
Tabla 9 - EndPoints Días de una dieta.....	12
Tabla 10 - EndPoints Comidas.....	12
Tabla 11 - EndPoints Ingredientes	13
Tabla 12 – EndPoints Ejercicios Globales.....	13
Tabla 13 - EndPoints Rutinas.....	14
Tabla 14 - EndPoints Días de una rutina.....	14
Tabla 15 - EndPoints Ejercicios del Usuario.....	14
Tabla 16 - EndPoints Series.....	15
Tabla 17 - EndPoints Registros diarios de comida.....	15
Tabla 18 - Pruebas del frontend	29

ÍNDICE DE FIGURAS

Ilustración 1 - Arquitectura (parcial) de la capa de presentación.....	10
Ilustración 2 - Diagrama componentes del BackEnd.....	16
Ilustración 3 - Dominio del Usuario.....	17
Ilustración 4 - Dominio de las dietas.....	18
Ilustración 5 - Dominio de las rutinas	20
Ilustración 6 - Ejemplo de Controller	21
Ilustración 7 - Service de los días de una rutina.....	22
Ilustración 8 - Ejemplo Entidad Usuario	23
Ilustración 9 - Interfaz Repository de la entidad Alimento	23
Ilustración 10 - Service de los Usuarios	24
Ilustración 11 - Clase Result.....	25
Ilustración 12 - Interfaz del perfil de un usuario.....	25
Ilustración 13 - Clase ViewModel del Login de los Usuarios	26
Ilustración 14 - Ejemplo prueba unitaria con objetos Mock	28
Ilustración 15 - Ejemplo prueba de integración	28
Ilustración 16 - Ejemplo prueba de rutinas en Postman	29
Ilustración 17 - Pantallas de inicio de sesión y de bienvenida.....	30
Ilustración 18 - Pantallas para la creación y consulta de rutinas de gimnasio..	31
Ilustración 19 - Pantallas para añadir y consultar ingredientes	31

RESUMEN

La cantidad de tecnologías, dispositivos y nuevas herramientas que han surgido estos últimos años de grandes avances en la informática han cambiado la vida de las personas por completo. Estos cambios han traído grandes beneficios que nos han permitido tener una mejor calidad de vida, pero en otros casos estas facilidades pueden ser un arma de doble filo que muchas veces no tenemos en cuenta.

Esto se puede ver reflejado en que actualmente y fijándonos en el futuro de un mundo cada vez más tecnológico la vida de las personas es cada vez más sedentaria y esto puede afectar negativamente a su salud. Un ejemplo muy claro es que cada vez hay más teletrabajo donde, desde la comodidad de su sofá o escritorio; cualquier persona puede trabajar sin tener que salir de casa.

Por todo lo mencionado anteriormente, la aplicación móvil que se presenta en este trabajo tiene como objetivo fomentar en las personas la necesidad de hacer ejercicio y llevar una vida más saludable ofreciéndoles facilidades a la hora de crear sus propias rutinas de gimnasio y almacenar sus dietas de forma automática para que cumplir sus metas y objetivos sea más fácil que nunca.

Para poder alcanzar el objetivo y desarrollar la aplicación se van a utilizar distintas tecnologías. En primer lugar, para la creación de la aplicación móvil se utilizará Android Studio, mientras que la información se gestionará a través de un servicio REST que se comunicará con una base de datos SQL donde se almacenarán todos los datos de la aplicación.

Palabras clave: Aplicación móvil, REST, salud, ejercicio, sedentarismo, dieta.

ABSTRACT

Nowadays, with all the technologies, devices, and new tools that have emerged in recent years and the great advances in computing, people's lives have changed completely. These changes have brought great benefits that have allowed us to have a better quality of life, but in other cases, these benefits can be a double-edged sword that we often overlook.

This can be seen in the fact that currently and looking to the future of an increasingly technological world, people's lives are becoming more sedentary, and this can negatively affect their health. A very clear example is the increasing trend of teleworking, where anyone can work from the comfort of their couch or desk without having to leave home.

For all the reasons mentioned above, this mobile application aims to encourage people to exercise and lead a healthier life by offering them the possibility to create their own gym routines and automatically store their diets so that achieving their goals and objectives is easier than ever.

To achieve this objective and develop the application, we will use different technologies. First, to create the app, Android Studio will be used and a REST service for the various system functions. A SQL database will be required to store all user information.

Keywords: Mobile app, REST, health, exercise, sedentary lifestyle, diet.

1. INTRODUCCIÓN

En este apartado se van a dar a conocer brevemente las razones que han conducido a la elección de este tema para la realización del Trabajo Fin de Grado (TFG) y cuáles son los objetivos que se pretenden alcanzar con su desarrollo.

1.1 Motivación

Desde la aparición de los primeros ordenadores hasta la actualidad la revolución tecnológica ha experimentado un crecimiento exponencial con grandes avances y la aparición de infinidad de invenciones que han hecho que la vida de las personas haya cambiado muy significativamente en un periodo muy corto de tiempo. Estos avances han supuesto una gran mejora en la calidad de vida de toda la sociedad, pero también existen algunos inconvenientes que muchas veces pueden pasar desapercibidos.

Como consecuencia de estos avances, en algunos casos, las personas han adoptado un estilo de vida cada vez más sedentario. El teletrabajo, la mensajería instantánea o las videollamadas, entre otras cosas, han reducido la necesidad de las personas de desplazarse físicamente. Actividades cotidianas como ese paseo de quince minutos hasta la oficina o tener que desplazarse en bicicleta hasta el trabajo cada día, que antes formaban parte de nuestra rutina diaria, nos aportaban beneficios en cuanto a la salud se refiere y esto tiende a desaparecer.

A todo esto hay que sumarle un factor fundamental en las personas y que lamentablemente nadie puede comprar, el tiempo. Muchas personas que intentan adoptar un estilo de vida más activo y comenzar una etapa más saludable en sus vidas acaban abandonándolo por todo el tiempo que les quita el hecho de tener que estar pendiente de crear sus rutinas de gimnasio, recordar qué ejercicio tiene que hacer hoy o cuántas proteínas tenía que comer en la cena para poder llegar a su objetivo.

Por todos estos motivos el objetivo de este TFG es desarrollar una aplicación fitness sencilla, intuitiva y eficiente que permita a las personas tener almacenada toda la información sobre sus rutinas, dietas y ejercicios y almacenar un histórico de sus avances y progresos.

1.2 Objetivos

El objetivo global de la aplicación es fomentar en las personas un estilo de vida más saludable ya sea haciendo rutinas de gimnasio, comenzando una dieta equilibrada o ambas cosas.

Este objetivo global se traduce en los siguientes subobjetivos para la aplicación móvil a desarrollar:

- Gestionar rutinas de gimnasio de manera fácil y sencilla.
- Almacenar dietas específicas para poder llevar un conteo de los nutrientes principales que cada persona tiene que consumir.
- Informar a las personas sobre las propiedades de los alimentos y los ejercicios más adecuados en función de los grupos musculares.

1.3 Organización de la memoria

Para la organización de la memoria se ha dividido la misma en diferentes capítulos que explican todo el proceso que se ha seguido para desarrollar la aplicación:

- En el Capítulo 2 se explican las distintas tecnologías y herramientas con las que se ha desarrollado la aplicación.
- En el Capítulo 3 se explican los diferentes requisitos funcionales y no funcionales que debemos lograr para completar la aplicación.
- En el Capítulo 4 se detalla la estructura en la que se divide la aplicación móvil.
- En el Capítulo 5 se explica cómo se han implementado cada una de las partes explicadas en la arquitectura y el diseño.
- En el Capítulo 6 se detallan las distintas pruebas con las que se verifica que la aplicación cumple su funcionamiento.
- En el Capítulo 7 se resumen las conclusiones que se han sacado a lo largo del desarrollo del proyecto y se detallan posibles trabajos futuros con los que se podría ir mejorando la aplicación.

2. TECNOLOGÍAS Y HERRAMIENTAS

En este capítulo se describen las distintas tecnologías, herramientas, lenguajes de programación y entornos de desarrollo que se han utilizado en el desarrollo de la aplicación.

2.1 Entornos de desarrollo

Para construir el código de la aplicación se han utilizado dos entornos de desarrollo distintos, uno para el backend y otro para el frontend de la aplicación:

- IntelliJ IDEA: Entorno utilizado para todo el desarrollo del backend. Combinado con el framework de Spring Boot se ha conseguido gestionar la API REST con eficiencia gracias al manejo de dependencias y herramientas que nos ofrece, aparte de mucho código básico autocompletado.
- Android Studio: Entorno utilizado para el desarrollo al completo del frontend. Debido a la decisión de realizar la aplicación en Android este es el entorno ideal para ello. Es muy útil para diseñar interfaces usando XML, además de su potente tecnología con la que podemos hacer múltiples pruebas con el dispositivo.

Además, se ha hecho uso de Visual Studio como entorno para el diseño de todos los diagramas de la documentación de la aplicación combinado con PlantUML.

Por último, cabe resaltar que se ha utilizado ChatGPT como soporte para completar código.

2.2 Lenguajes y frameworks

Los lenguajes y/o frameworks utilizados como base para la implementación del código han sido:

- Java: Para el desarrollo del código tanto del backend como del frontend se ha escogido trabajar con el lenguaje Java. Es un lenguaje orientado a objetos, siendo este uno de los lenguajes más conocidos de todos. Por todo esto es un lenguaje ideal para construir una aplicación segura y estable.
- Spring Boot: Es un framework de Java utilizado para desarrollar APIs REST de forma rápida y eficiente, que nos ayuda a automatizar gran parte de la configuración del servicio.

2.3 Diseño y Pruebas

- PlantUML: Herramienta utilizada para el diseño de todos los diagramas ya sean de clases, arquitectura o de componentes. Esto permite crear los diseños de nuestra aplicación de forma clara y sencilla para después programarlos.
- Postman: Esta plataforma se ha utilizado en la fase de pruebas para comprobar si el funcionamiento de los servicios del backend eran correctos antes de utilizarlos en el frontend. Permite utilizar distintos formatos como JSON o XML, muestra los códigos HTTP de respuesta y permite agregar cuerpo a las peticiones si el servicio lo requiere.

2.4 Comunicación y gestión de datos

- MySQL: Para la gestión de la base de datos relacional se ha escogido MySQL ya que permite mantener la persistencia e integridad de los datos de nuestros usuarios y los mantienen seguros.
- Retrofit: Es una librería de Android Studio con la cual se comunica el backend con el frontend a través de peticiones HTTP facilitando la comunicación entre ambas partes.

2.5 Gestión del proyecto

- ScrumDesk: Esta plataforma se ha utilizado para la gestión del desarrollo de la aplicación. Bajo un desarrollo tipo Scrum usando una metodología de trabajo ágil se ha creado el Product Backlog con las distintas historias de usuario que se han implementado. Además de la planificación de los distintos Sprints y el seguimiento del progreso que se ha ido realizando viendo qué historias están hechas y cuáles deben desarrollarse todavía.
- GitHub: Esta plataforma se ha usado para almacenar el código de la aplicación. Cada vez que se han hecho grandes cambios en el código se han ido subiendo a GitHub. Gracias a esta plataforma, además, se han creado distintas ramas con las que tener distintas versiones del código manteniendo siempre la coherencia y seguridad en él.

Todo el código completo del proyecto se puede consultar en el siguiente repositorio de GitHub: <https://github.com/AlejandroAranaGomez/TFG>

3. ANÁLISIS DE REQUISITOS

En este apartado vamos a detallar los requisitos necesarios para que el sistema funcione correctamente. En primer lugar, definiremos los requisitos funcionales y después continuaremos con los requisitos no funcionales.

3.1 Requisitos funcionales

Los requisitos funcionales definen las funciones que el sistema tiene que ser capaz de hacer para satisfacer las necesidades del usuario. En este caso se han dividido estos requisitos en tres grandes grupos. Por un lado, requisitos generales que toda aplicación suele tener como registro, inicio de sesión o notificaciones. Por otro tenemos requisitos referidos a la alimentación y la creación de dietas. Por último, tenemos los requisitos relativos al tema del gimnasio con los ejercicios y la posibilidad de crear rutinas.

En cuanto a la técnica elegida para esta definición de requisitos se ha decidido usar historias de usuario. Esta técnica es muy frecuentemente utilizada en metodologías ágiles de software donde los requisitos se describen desde el punto de vista del usuario, el cual expresa el resultado final que desea y las necesidades que él requiere que se cumplan para estar satisfecho.

Estas necesidades se muestran divididas en tres tablas diferentes. En la Tabla 1 encontramos la descripción de las historias de usuario generales que tienen que ver con la gestión de usuarios. La Tabla 2 recoge todos los requisitos que la aplicación debe alcanzar para cubrir las necesidades de los usuarios en lo relativo a dietas. Por último, en la Tabla 3 se muestra todo lo relacionado con las rutinas de gimnasio.

Tabla 1 - Requisitos generales del sistema

Identificador	Descripción
RG-01	Yo, como usuario, quiero poder crear un perfil de usuario en la aplicación en el que indicar mis objetivos y cualidades para que pueda alcanzar mis objetivos.
RG-02	Yo, como usuario, quiero poder iniciar sesión sobre un perfil ya registrado utilizando el nombre y la contraseña que establecí en el registro, para poder ver mi progreso guardado.
RG-03	Yo, como usuario, quiero poder modificar mi perfil en cualquier momento para poder detallar cuales son mis objetivos y mis cualidades físicas o cambiar cualquiera de mis datos personales.
RG-04	Yo, como usuario, quiero poder actualizar cada cierto tiempo mi peso actual y tener un historial de todos mis pesos con el objetivo de poder comprobar si estoy cumpliendo mi objetivo.

Tabla 2 - Requisitos sobre dietas

Identificador	Descripción
RA-01	Yo, como usuario, quiero poder ver las propiedades del alimento que seleccione (grasas, carbohidratos, calorías, etc.) para evaluar si ese alimento es adecuado para mi dieta.
RA-02	Yo, como usuario, quiero poder anotar las comidas que voy realizando al día para poder tener anotadas las grasas, calorías, proteínas y carbohidratos que voy consumiendo a lo largo del día para comprobar que cumplo mis objetivos.
RA-03	Yo, como usuario, quiero poder recibir notificaciones a modo de recordatorio para motivarme a cumplir mi objetivo.
RA-04	Yo, como usuario, quiero poder buscar cualquier alimento a través de un buscador para poder informarme de sus propiedades alimenticias.
RA-05	Yo, como usuario, quiero poder crear mis propios alimentos personalizados para anotar las propiedades de los alimentos que consumo en mi día a día para así llevar cuenta con exactitud de las calorías que consumo.
RA-06	Yo, como usuario, quiero poder modificar los alimentos que previamente he creado para cambiar todos los atributos que quiera (nombre, calorías, proteínas, grasas y carbohidratos).
RA-07	Yo, como usuario, quiero poder borrar cualquier alimento que previamente había creado para que no me moleste en el caso de que ya no lo vaya a usar más.
RA-08	Yo, como usuario, quiero poder crear una dieta propia para poder organizar todos los alimentos que tengo que comer a lo largo de la semana y saber todas las calorías, grasas, proteínas y carbohidratos que consumo.
RA-09	Yo, como usuario, quiero poder editar el nombre y descripción de una dieta previamente creada para tener la información siempre actualizada.
RA-10	Yo, como usuario, quiero poder borrar una dieta si ya no la voy a utilizar más para no ocupar espacio innecesario.
RA-11	Yo, como usuario, quiero poder crear un día de toda la semana en mi propia dieta donde almacenar qué comidas tengo que hacer en ese día para así alcanzar a mi objetivo.
RA-12	Yo, como usuario, quiero poder cambiar el nombre de un día de una dieta para utilizar nombres más precisos y adecuados a mi dieta.
RA-13	Yo, como usuario, quiero poder borrar cualquier día de mi dieta si es que esta dieta ha cambiado para tener mi dieta actualizada en todo momento.
RA-14	Yo, como usuario, quiero poder crear una comida dentro de un

	día de mi dieta para almacenar ahí todos los alimentos que voy a consumir.
RA-15	Yo, como usuario, quiero poder editar una comida dentro de un día de mi dieta en caso de que quiera cambiarle el nombre a esa comida para tener la información siempre actualizada.
RA-16	Yo, como usuario, quiero poder borrar una comida de un día si me he equivocado o ya no voy a usar más esa comida para no ocupar espacio innecesario.
RA-17	Yo, como usuario, quiero poder añadir un ingrediente a una comida para tener almacenada la cantidad y los diferentes nutrientes que consumo.
RA-18	Yo, como usuario, quiero poder editar la cantidad de gramos que consumo de un ingrediente en una comida en caso de que cambie la cantidad de comida que consumo para tener siempre mi dieta actualizada.
RA-19	Yo, como usuario, quiero poder borrar un ingrediente de una comida si por ejemplo mi dieta ha cambiado para no tener ingredientes que no voy a comer.

Tabla 3 - Requisitos sobre rutinas de gimnasio

Identificador	Descripción
RR-01	Yo, como usuario, quiero poder tener acceso a imágenes que me muestren cómo realizar correctamente la técnica de los distintos ejercicios.
RR-02	Yo, como usuario, quiero poder buscar entre todos los ejercicios filtrando por el músculo que esté interesado para poder buscarlo en ese momento.
RR-03	Yo, como usuario, quiero poder crear una rutina de gimnasio con todos los ejercicios que quiero realizar a lo largo de la semana para saber qué tengo que hacer cuando vaya al gimnasio.
RR-04	Yo, como usuario, quiero poder editar el nombre y el resumen de una rutina de gimnasio para tener toda la información siempre actualizada.
RR-05	Yo, como usuario, quiero poder eliminar una rutina propia que ya no voy a volver a usar para no ocupar espacio innecesario.
RR-06	Yo, como usuario, quiero poder crear un día específico dentro de una rutina de gimnasio para organizar mi semana de entrenamiento.
RR-07	Yo, como usuario, quiero poder editar el nombre de un día concreto de una rutina de gimnasio para identificar sobre qué entrenamos cada día.

RR-08	Yo, como usuario, quiero poder borrar un día concreto de una rutina de gimnasio si ya no voy a usarlo más para no tener días innecesarios.
RR-09	Yo, como usuario, quiero poder añadir un ejercicio a un día específico para saber qué ejercicios debo hacer cada día.
RR-10	Yo, como usuario, quiero poder borrar un ejercicio de un día en el caso en el que cambie mi rutina para no ocupar espacio innecesario.
RR-11	Yo, como usuario, quiero poder crear una serie en mi ejercicio para tener anotado el peso y las repeticiones que soy capaz de hacer.
RR-12	Yo, como usuario, quiero poder actualizar mis series cada vez que haga una serie para tener siempre la información al día.
RR-13	Yo, como usuario, quiero poder borrar una serie si voy a cambiar la forma de hacer el ejercicio y no necesito hacer tantas series para tener la rutina siempre actualizada.

3.2 Requisitos no funcionales

Además de los requisitos funcionales se han de definir también los requisitos no funcionales. En este apartado se explican las condiciones que deben cumplir ciertas características del sistema para su correcto funcionamiento. Todos estos requisitos están recogidos en la Tabla 4.

Tabla 4 - Requisitos No Funcionales

Identificador	Descripción	Categoría
RN-01	El sistema debe ser capaz de garantizar la protección de datos de las contraseñas de todas las cuentas de cada uno de los usuarios.	Seguridad
RN-02	La base de datos del sistema debe garantizar la integridad y consistencia de los datos para evitar pérdidas de información.	Integridad y Disponibilidad
RN-03	La aplicación deberá funcionar para la versión de Android 8.0 (API 26) o superior.	Compatibilidad
RN-04	La aplicación debe ser sencilla y rápida de utilizar para que el usuario sienta que es una ayuda para su desarrollo personal y pueda cumplir sus objetivos.	Usabilidad

4. ARQUITECTURA Y DISEÑO

En este apartado se describen y detallan la arquitectura y el diseño que se han establecido para el desarrollo de la aplicación. Para este proyecto en concreto se ha utilizado una arquitectura cliente-servidor con tres capas distintas las cuales cada una tiene distintas funciones.

4.1 Arquitectura del sistema

Como se ha indicado anteriormente la aplicación sigue una arquitectura cliente-servidor, esto significa que el cliente realiza peticiones al servidor para obtener información o poder usar ciertos servicios. Por otro lado, el servidor es el encargado de procesar las peticiones del cliente para generar respuestas que satisfagan las necesidades de dicho cliente.

Las tres capas en las que se divide el proceso de intercambio de información serían las siguientes: capa de presentación, capa de negocio y capa de datos. A continuación, se explica en detalle cada una de ellas.

4.2 Capa de presentación

La primera capa es la capa de presentación, es la encargada de mostrar la interfaz al usuario, recoger las acciones que este realice en la aplicación y enviarlas a la capa de negocio para que sean procesadas y transformadas en una respuesta que se le devolverá al usuario.

Esta capa la implementa la aplicación móvil y para su desarrollo se ha utilizado el entorno de desarrollo de Android Studio con el que se pueden crear aplicaciones Android.

Además, para seguir unas buenas prácticas de desarrollo se han aplicado en su diseño dos patrones de arquitectura: el MVVM (Model-View-ViewModel) [1] y el patrón Repository [11].

En el patrón MVVM distinguimos tres partes diferenciadas:

- El modelo (Model) es la parte de la aplicación donde se identifican las entidades (usuario, dietas, rutinas, etc. en este caso).
- La vista (View) es la interfaz con la que el usuario interactúa para obtener la información y comunica el Model con el ViewModel. Su función es mostrar los datos y recoger las acciones del usuario.
- Por último, tenemos el ViewModel que conecta el View con el Model. Su función es obtener los datos del modelo y transformarlos para que sean mostrados en la vista.

El Repository es el responsable de abstraer el modo en que se accede al Model. En este proyecto en particular las entidades del modelo se obtienen a través de un servicio REST. Para poder conectar este servicio REST con el frontend se utilizará la librería Retrofit.

En la Ilustración 1 se muestra, a modo de ejemplo, una parte de la arquitectura de la capa de presentación. La capa View del patrón MVVM se implementa mediante las Activities y Fragments Android que muestran la interfaz al usuario. En este caso se ha extraído la parte referente a la autenticación del usuario con sus dos formas, iniciando sesión que sería el login o registrándose con el registro. El diagrama completo se puede ver en el repositorio de GitHub.

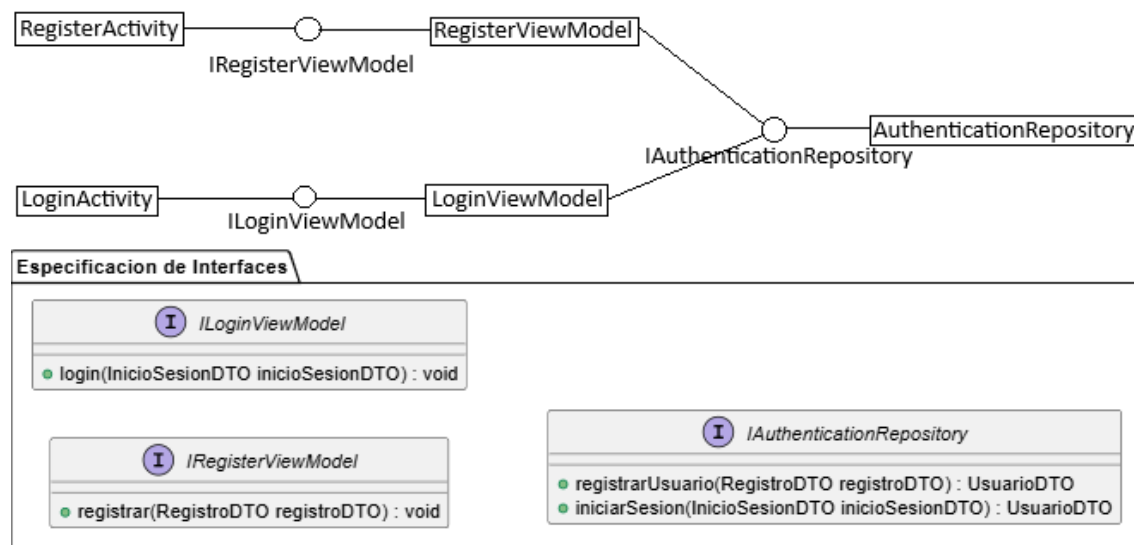


Ilustración 1 - Arquitectura (parcial) de la capa de presentación

4.3 Capa de negocio

Esta es la capa que se encarga de implementar la lógica de la aplicación, gestionar errores y realizar los cálculos necesarios para todas las propiedades de las dietas (calorías, grasas, carbohidratos y proteínas).

En este caso para el desarrollo de la capa de negocio se ha seguido el modelo de servicio REST (Representational State Transfer) [10]. Este servicio REST implementa la comunicación entre la capa de negocio y la capa de presentación a través de peticiones HTTP. Estas peticiones acceden a recursos que representan las entidades de nuestro dominio de datos donde cada recurso tiene su propio URI con el que se identifica. Estos recursos se pueden manipular con los métodos GET, POST, PUT y DELETE. A continuación se muestra el diseño del servicio REST a través de una serie de tablas con los recursos y endpoints definidos para la gestión de la aplicación.

En la Tabla 5 se muestran los endpoints referentes al recurso Usuario.

Tabla 5 - EndPoints Usuarios

Recurso	URI	Método	Código HTTP
Usuarios	/api/usuarios	POST	201 Created 409 Conflict
	/api/usuarios/token	POST	200 OK 401 Unauthorized
Usuario	/api/usuarios/{idUserio}	GET	200 OK 404 Not Found
		PUT	200 OK 404 Not Found
Historial Peso	/api/usuarios/{idUserio} /historialPeso	GET	200 OK 404 Not Found
	/api/usuarios/{idUserio} /historialPeso	POST	201 Created 404 Not Found

En la Tabla 6 se muestran los endpoints referentes al recurso Alimentos Globales mientras que en la Tabla 7 se muestran los del recurso Alimentos del Usuario.

Tabla 6 - EndPoints Alimentos Globales

Recurso	URI	Query Params	Método	Código HTTP
Alimentos Globales	/api/alimentos	alimento	GET	200 OK

Tabla 7 - EndPoints Alimentos del Usuario

Recurso	URI	Método	Código HTTP
Alimentos del Usuario	/api/usuarios /{idUserio}/alimentos/	GET	200 OK 404 Not Found
		POST	201 Created 404 Not Found 409 Conflict 400 Bad Request
Alimento del Usuario	/api/usuarios /{idUserio}/alimentos/ {idAlimento}	PUT	200 OK 404 Not Found 409 Conflict 400 Bad Request 403 Forbidden
		DELETE	200 Ok 404 Not Found 403 Forbidden

En la Tabla 8 se muestran los endpoints referentes al recurso Dieta mientras que en la Tabla 9 los referentes al recurso Día en Dieta.

Tabla 8 - EndPoints Dietas

Recurso	URI	Método	Código HTTP
Dietas del Usuario	/api/usuarios/{idUserio}/dietas	GET	200 OK 404 Not Found
		POST	201 Created 404 Not Found 409 Conflict
Dieta del Usuario	/api/usuarios/{idUserio}/dietas/{idDieta}	PUT	200 OK 404 Not Found 409 Conflict 403 Forbidden
		DELETE	200 Ok 404 Not Found 403 Forbidden
		GET	200 OK 404 Not Found 403 Forbidden
	/api/usuarios/{idUserio}/dietas/{idDieta}/estado	PUT	200 OK 404 Not Found 409 Conflict

Tabla 9 - EndPoints Días de una dieta

Recurso	URI	Método	Código HTTP
Día de una dieta	/api/usuarios/{idUserio}/dietas/{idDieta}/{diaSemana}	PUT	200 OK 201 Created 404 Not Found 403 Forbidden
		DELETE	200 OK 404 Not Found 403 Forbidden

En la Tabla 10 se muestran los endpoints referentes al recurso Comida.

Tabla 10 - EndPoints Comidas

Recurso	URI	Método	Código HTTP
Comidas de un día en dieta	/api/usuarios/{idUserio}/dietas/{idDieta}/{diaSemana}/comidas	GET	200 OK 404Not Found
		POST	201 Created 404 Not Found 409 Conflict

Comida de un día en dieta	/api/usuarios/{idUserario} /dietas/{idDieta}/{diaSemana} /comidas/{idComida}	PUT	200 OK 404 Not Found 403 Forbidden 409 Conflict
		DELETE	200 OK 404 Not Found 403 Forbidden

En la Tabla 11 se muestran los endpoints referentes al recurso Ingrediente.

Tabla 11 - EndPoints Ingredientes

Recurso	URI	Método	Código HTTP
Ingredientes de una comida	/api/usuarios/{idUserario} /dietas/{idDieta} /{diaSemana}/comidas/ {idComida}/ingredientes	GET	200 OK 404 Not Found
		POST	201 Created 404 Not Found 400 Bad Request 409 Conflict
Ingrediente de una comida	/api/usuarios/{idUserario} /dietas/{idDieta} /{diaSemana}/comidas/ {idComida}/ingredientes/ {idIngrediente}	PUT	200 OK 404 Not Found 403 Forbidden 400 Bad Request
		DELETE	200 OK 404 Not Found 403 Forbidden

La Tabla 12 muestra los endpoints referentes al recurso Ejercicios Globales.

Tabla 12 – EndPoints Ejercicios Globales

Recurso	URI	Método	Código HTTP
Ejercicios Globales	/api/ejercicios	GET	200 OK 500 Internal Server Error

En la Tabla 13 se muestran los endpoints referentes al recurso Rutinas y en la Tabla 14 los del recurso Día en Rutina.

Tabla 13 - EndPoints Rutinas

Recurso	URI	Método	Código HTTP
Rutinas del Usuario	/api/usuarios/{idUserio}/rutinas	GET	200 OK 404 Not Found
		POST	201 Created 404 Not Found 409 Conflict
Rutina del Usuario	/api/usuarios/{idUserio}/rutinas/{idRutina}	PUT	200 OK 404 Not Found 409 Conflict 403 Forbidden
		DELETE	200 Ok 404 Not Found 403 Forbidden
		GET	200 OK 404 Not Found 403 Forbidden

Tabla 14 - EndPoints Días de una rutina

Recurso	URI	Método	Código HTTP
Día de una rutina	/api/usuarios/{idUserio}/rutinas/{idRutina}/{diaSemana}	PUT	200 OK 201 Created 404 Not Found 403 Forbidden
		DELETE	200 OK 404 Not Found 403 Forbidden

La Tabla 15 muestra los endpoints referentes al recurso Ejercicios del Usuario.

Tabla 15 - EndPoints Ejercicios del Usuario

Recurso	URI	Método	Código HTTP
Ejercicios de un día en rutina	/api/usuarios/{idUserio}/rutinas/{idRutina}/{diaSemana}/ejercicios	GET	200 OK 404 Not Found
		POST	201 Created 404 Not Found 409 Conflict

Ejercicio de un día en rutina	/api/usuarios/{idUserario} /rutinas/{idRutina} /{diaSemana}/ejercicios/{idEjercicio}	DELETE	200 OK 404 Not Found 403 Forbidden
-------------------------------	--	--------	--

En la Tabla 16 se muestran los endpoints referentes al recurso Series.

Tabla 16 - EndPoints Series

Recurso	URI	Método	Código HTTP
Series de un ejercicio	/api/usuarios/{idUserario} /rutinas/{idRutina} /{diaSemana}/ejercicios/{idEjercicioEnDiaRutina}/series	GET	200 OK 404 Not Found
		POST	201 Created 404 Not Found 400 Bad Request 409 Conflict
Serie de un ejercicio	/api/usuarios/{idUserario} /rutinas/{idRutina} /{diaSemana}/ejercicios/{idEjercicioEnDiaRutina}/series/{idSerie}	PUT	200 OK 404 Not Found 403 Forbidden 400 Bad Request
		DELETE	200 OK 404 Not Found 403 Forbidden

La Tabla 17 muestra los endpoints referentes al recurso Comida Diaria.

Tabla 17 - EndPoints Registros diarios de comida

Recurso	URI	Método	Código HTTP
Comidas de hoy del usuario	/api/seguimiento/{idUserario}	GET	200 OK 404 Not Found
Comida de hoy del usuario	/api/seguimiento/{idUserario} /comida/{idComida}	POST	200 OK 404 Not Found 409 Conflict
		DELETE	200 OK 404 Not Found

Para el diseño interno de esta capa se ha seguido el patrón arquitectural CSR (Controller-Service-Repository):

- El controller es el encargado de recibir las solicitudes de la capa de presentación a través del uso de endpoints que sirven de acceso a los recursos de la API.
- El service se encarga de mantener la lógica de negocio en la que se realizan todas las operaciones para calcular que toda la información sea correcta y los datos mantengan la consistencia.
- El repository conecta la capa de negocio con la capa de datos, es decir, conecta el backend con la base de datos donde se almacena toda la información de nuestra aplicación.

En la Ilustración 2 se puede ver un fragmento del diagrama de componentes del backend, en este caso, referente al recurso Usuario y a la autenticación en el sistema. Además, se muestran las distintas interfaces de los componentes Service y Repository que se han creado para las clases del usuario, el historial del peso y las credenciales. Se muestra solo una parte debido a que el diagrama completo es muy extenso. Dicho diagrama se puede encontrar completo en el repositorio del proyecto en GitHub.

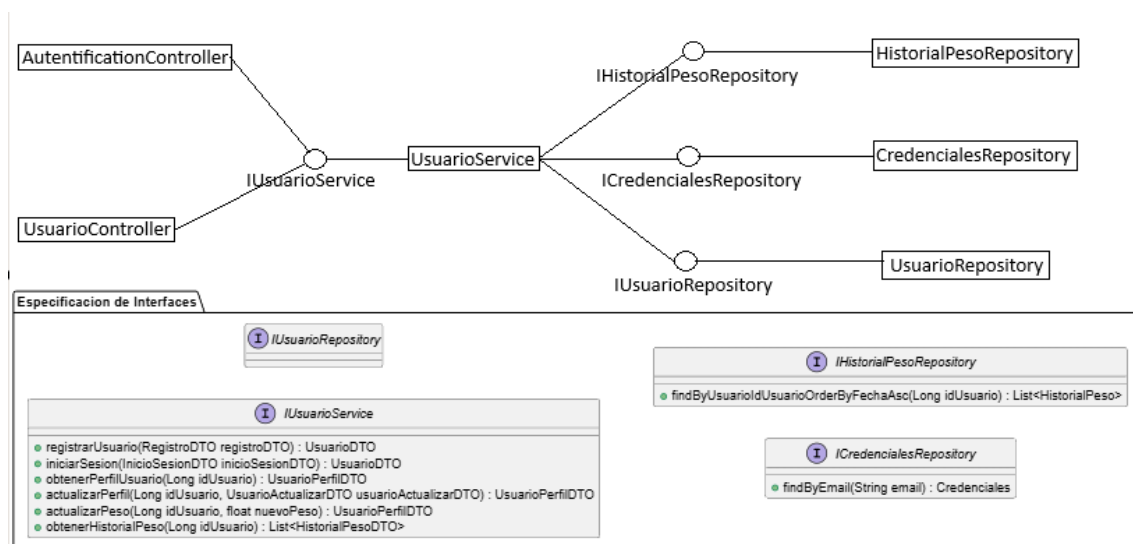


Ilustración 2 - Diagrama componentes del BackEnd

4.3.1 Modelo de dominio

En este apartado se muestra el modelo de dominio utilizado para representar toda la información de la aplicación. Este modelo recoge todas las entidades y las relaciones que estas entidades tienen entre ellas. Es muy útil para entender cómo se organiza la aplicación y cómo es el flujo de datos e información. Ya que el diagrama de clases correspondiente es demasiado grande para mostrarlo en una sola imagen se va a dividir en tres grandes apartados diferenciados: datos del usuario, datos de las dietas y datos de las rutinas.

Datos del usuario

En este apartado se muestra la información relativa a los usuarios. Como se observa en el diagrama de clases de la Ilustración 3, por un lado, tenemos al usuario con todos los datos necesarios para poder crear su cuenta al registrarse. Aparte se guardan sus credenciales, en este caso email y contraseña, con los que posteriormente puede autenticarse en la aplicación para acceder a su cuenta. Para mantener la seguridad de los usuarios estas contraseñas se almacenan de forma encriptada.

Por último, por cada usuario se almacena su historial de peso, donde se almacenan todos los pesos que el usuario ha ido anotando en la aplicación para que pueda llevar un seguimiento del progreso que está teniendo a lo largo del tiempo.



Ilustración 3 - Dominio del Usuario

Datos de las dietas

En la Ilustración 4 se aprecia la relación de los datos de un usuario con sus dietas. En la aplicación existen alimentos tanto generales, que un usuario puede consultar en un buscador para acceder a sus propiedades, como alimentos personalizados, que el usuario puede crear, asignando los valores nutricionales que quiera.

Además están las dietas, divididas en días, donde cada día tiene sus propias comidas con sus ingredientes. Estas dietas muestran las proteínas, grasas, carbohidratos y calorías que el usuario tiene que consumir al día.

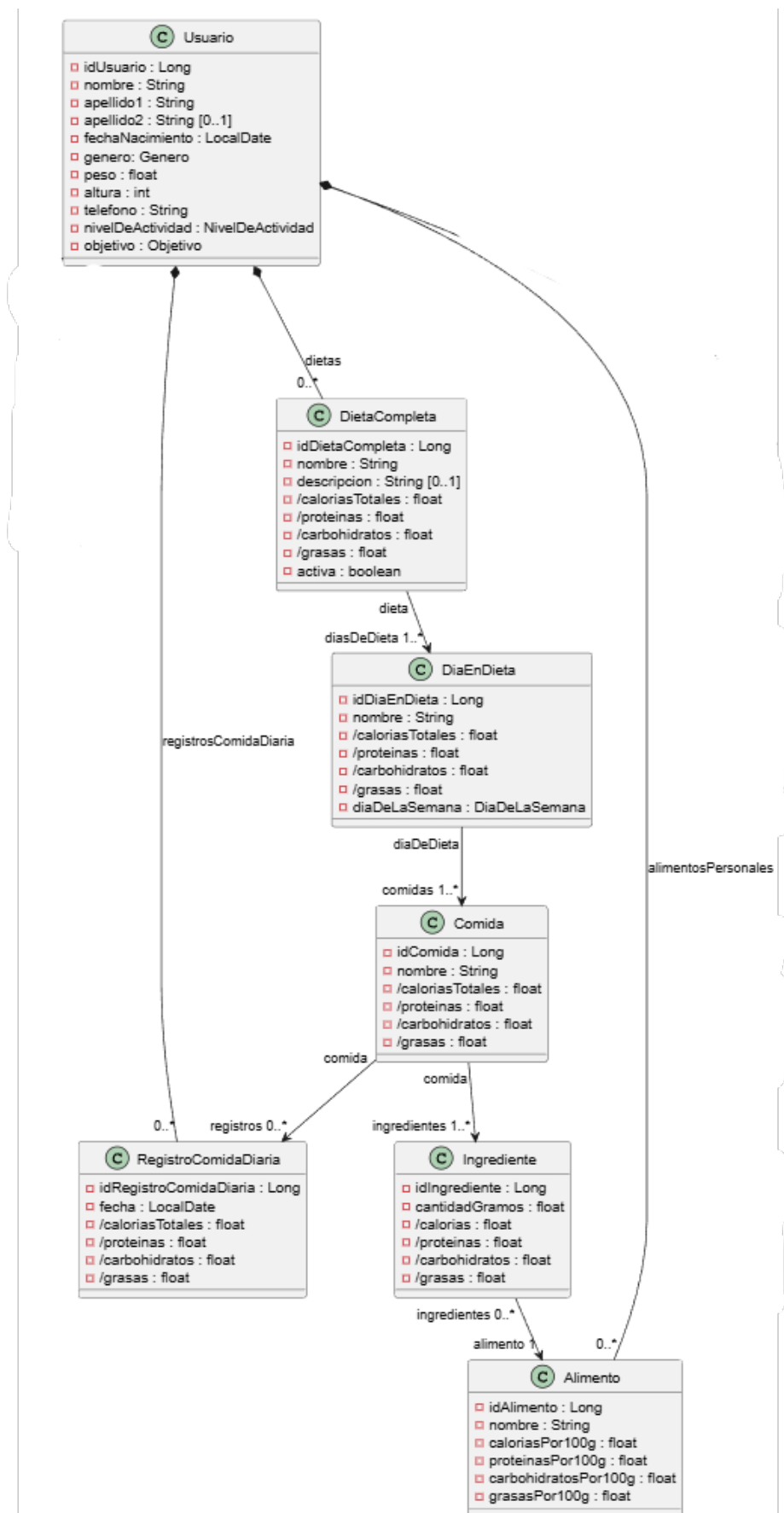


Ilustración 4 - Dominio de las dietas

Además, se pueden almacenar las comidas diarias para que cada día, cuando el usuario realice una comida, este lo almacene en la aplicación y pueda realizar un seguimiento de cuantas proteínas, grasas, carbohidratos y calorías le faltan por consumir ese día.

Datos de las rutinas

Por último tenemos los datos referentes a las rutinas, que se muestran en el diagrama de la Ilustración 5. En este apartado tenemos, por un lado, los ejercicios de gimnasio que un usuario puede consultar en un buscador y filtrar por los distintos grupos musculares. Además, el usuario puede crear rutinas de gimnasio divididas en los distintos días de la semana. Cada día el usuario puede añadir los ejercicios que él quiera y dentro de esos ejercicios cuando esté haciendo series en el gimnasio, puede añadirlas al ejercicio para llevar el seguimiento de los pesos y las repeticiones que puede hacer y así ver su progreso.

4.4 Capa de datos

Por último, tenemos la capa de datos en la que se almacena la información de todos nuestros usuarios. El objetivo de esta capa es mantener la persistencia de los datos de forma segura. Para alcanzar este objetivo se utiliza una base de datos en MySQL donde para cada entidad tenemos una tabla que almacena todas las instancias creadas de esa misma entidad.

En este proyecto se ha utilizado una base de datos relacional con un modelo Entidad-Relación en el que cada entidad es una clase y las relaciones hacen que las clases se comuniquen entre ellas. Para mapear las entidades con la base de datos se utiliza JPA (Java Persistence API) [11] que nos ayuda a automatizar este proceso de forma sencilla y eficiente manteniendo la persistencia de los datos.

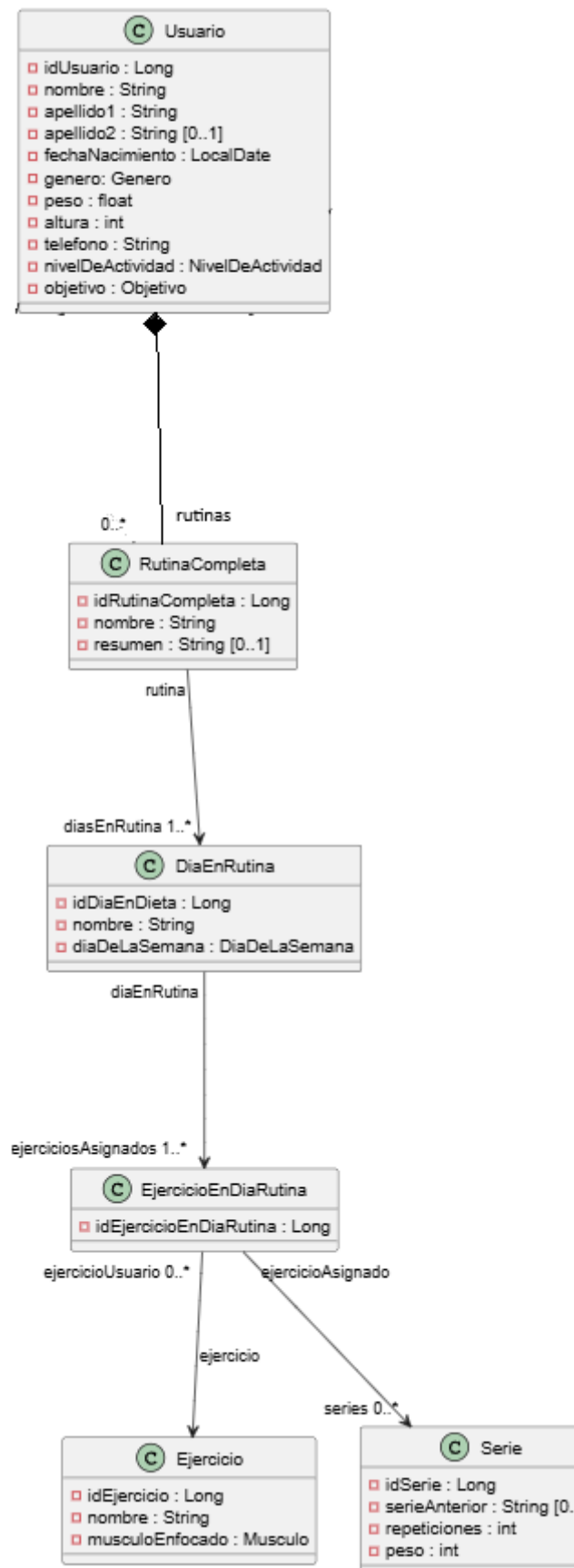


Ilustración 5 - Dominio de las rutinas

5. IMPLEMENTACIÓN

En este capítulo se describe el proceso que se ha seguido para implementar la arquitectura y el diseño que se han descrito en el capítulo anterior.

5.1 Implementación del backend

Para desarrollar esta parte de la aplicación y conseguir que todos los servicios funcionen correctamente y puedan ser utilizados en el frontend se ha utilizado el framework Spring Boot. Además, utilizando la arquitectura de tres capas CSR (Controller – Service - Repository) conseguimos separar cada una de las responsabilidades.

5.1.1 Capa de controladores

Esta capa es la encargada de gestionar las solicitudes HTTP utilizadas para acceder a los recursos y obtener la información correspondiente. Para ello se hace uso de URIs donde cada una se mapea a un método concreto que realiza una función específica.

En la Ilustración 6 se puede ver, a modo de ejemplo, un fragmento de uno de los Controller que se han creado en la aplicación. En este caso se puede apreciar la asignación del URI a uno de los métodos. También se aprecia la utilización de inyección de dependencias basada en constructor para que estos Controller puedan acceder a los Service. Además de los métodos se utilizan distintas anotaciones de Spring Boot:

- `@RestController`: Indica que la clase utiliza REST para manejar las peticiones.
- `@RequestMapping`: Indica la URI que se aplica a todos los métodos de la clase.
- `@GetMapping`: Indica qué tipo de petición HTTP corresponde al método al que está asociado y puede contener también el resto de su URI específica.

```
@RestController  Alejandro Arana
@RequestMapping("/api/usuarios")
public class UsuarioController {

    private UsuarioService usuarioService;  5 usages

    public UsuarioController(UsuarioService usuarioService) { this.usuarioService = usuarioService; }

    @GetMapping("/{idUsuario}")  Alejandro Arana
    public ResponseEntity<> obtenerPerfil(@PathVariable Long idUsuario) {
```

Ilustración 6 - Ejemplo de Controller

5.1.2 Capa de Servicios

Esta capa es la encargada de implementar la lógica de negocio y mantener la integridad de los datos de la aplicación. Las clases que la componen están anotadas con `@Service` que es una etiqueta de Spring Boot que indica que esa clase funciona como un Service. También debe comprobar que se cumplen las reglas de negocio, por ejemplo, un usuario no puede editar una rutina que no le pertenece. En la Ilustración 7 se puede ver la cabecera del service encargado de gestionar a los días de una rutina y como se usa inyección de dependencias para conseguir acceso a los componentes de la capa Repository.

```
@Service  ⚙ Alejandro Arana *
public class DiaEnRutinaService {

    private final RutinaCompletaRepository rutinaCompletaRepository; 3 usages
    private final DiaEnRutinaRepository diaEnRutinaRepository; 5 usages
    private final DiaEnRutinaAssembler diaEnRutinaAssembler; 2 usages

    public DiaEnRutinaService(DiaEnRutinaRepository diaEnRutinaRepository, no usa
                             RutinaCompletaRepository rutinaCompletaRepository,
                             DiaEnRutinaAssembler diaEnRutinaAssembler) {
        this.diaEnRutinaRepository = diaEnRutinaRepository;
        this.rutinaCompletaRepository = rutinaCompletaRepository;
        this.diaEnRutinaAssembler = diaEnRutinaAssembler;
    }
}
```

Ilustración 7 - Service de los días de una rutina

5.1.3 Capa de Repositorios

Esta capa está directamente conectada con capa de datos. Para lograr que los datos se transmitan entre capas se utilizan interfaces Repository donde además de las operaciones CRUD básicas para gestionar las entidades se han creado operaciones propias que posteriormente se han utilizado para que toda la lógica de los endpoints funcione correctamente.

Además, gracias a JPA el repositorio se encarga de realizar el mapeo Objeto-Relacional (ORM), es decir, traduce los objetos de las entidades Java a filas que se almacenan en las tablas de la base de datos MySQL.

En la Ilustración 8 se puede ver un fragmento de una entidad de la aplicación, en este caso la entidad Alimento, con algunas de las anotaciones típicas de JPA, como `@Entity`, `@Id` o `@Column`.

En la Ilustración 9 se muestra como ejemplo el Repository que gestiona los alimentos de nuestra aplicación. Podemos ver que se usa Spring Data JPA al extender de la interfaz `JpaRepository`, la cual da acceso a las operaciones CRUD que nombramos anteriormente.

```

@Entity
@Table(name = "alimento")
public class Alimento {

    // Clave Primaria BBDD
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idAlimento;

    @Column(nullable = false)
    private String nombre;
    @Column(nullable = false)
    private float calorías;
    @Column(nullable = false)
    private float proteínas;
}

```

Ilustración 8 - Ejemplo Entidad Usuario

Además, se han añadido dos métodos adicionales para utilizarlos en la capa de servicio. A estos dos métodos se les da nombres específicos dependiendo lo que queramos obtener de la base de datos. Por ejemplo, al usar “findByNombre” Spring Data JPA ya sabe que la consulta SQL que debe hacer utiliza el atributo nombre del Alimento como parámetro.

```

@Repository 3 usages alex
public interface AlimentoRepository extends JpaRepository<Alimento, Long> {

    // alimento de un usuario
    Optional<Alimento> findByNombreAndUsuario(String nombre, Usuario usuario);

    // devuelve los alimento globales y los del usuario.
    List<Alimento> findByUsuarioOrUsuarioIsNull(Usuario usuario); 1 usage alex
}

```

Ilustración 9 - Interfaz Repository de la entidad Alimento

5.2 Implementación del frontend

Esta parte de la aplicación es la responsable del desarrollo de la aplicación móvil. Para ello se ha escogido el sistema operativo de Android y el entorno de desarrollo se ha llevado a cabo en Android Studio, utilizando el lenguaje Java. Con todo esto para que la aplicación sea mantenible y escalable se ha seguido el patrón MVVM (Model – View - ViewModel) de forma que exista una separación entre las responsabilidades.

5.2.1 Capa Model

Esta es la capa que conecta el frontend con el backend. Está compuesta de varias clases que trabajan entre sí para conseguir acceder a la información que posteriormente se muestra y utiliza en la aplicación. En primer lugar, tenemos las clases POJOs (Plain Old Java Object), en este caso son las clases DTO

que representan los datos de la aplicación para manipularlos dentro de ella. Estas entidades se obtienen del servicio REST.

Para poder acceder a esta información del servicio REST se utiliza el patrón Repository de manera que se abstraen la forma en la que se accede a los datos del resto de la aplicación para que no tenga que conocer la comunicación con el backend. En este caso, el acceso se realiza usando la librería Retrofit. Las clases Repository utilizan interfaces Service que definen los endpoints del servicio REST, y sirven para enviar las peticiones HTTP y recibir las respuestas correspondientes. En la Ilustración 10 se puede ver un ejemplo de una interfaz Service. En ella se observa la definición de las URIs de los servicios del backend y las anotaciones que definen los parámetros que utilizan los métodos.

```
public interface UsuarioService {

    1 usage
    @GET("api/usuarios/{idUser}")
    Call<UsuarioPerfilDTO> obtenerPerfil(@Path("idUser") Long idUsuario);

    1 usage
    @PUT("api/usuarios/{idUser}")
    Call<UsuarioPerfilDTO> actualizarPerfil(@Path("idUser") Long idUsuario, @Body UsuarioActualizarDTO dto);

    1 usage
    @GET("api/usuarios/{idUser}/historialPeso")
    Call<List<HistorialPesoDTO>> obtenerHistorialPeso(@Path("idUser") Long idUsuario);

    1 usage
    @POST("api/usuarios/{idUser}/historialPeso")
    Call<UsuarioPerfilDTO> actualizarPeso(@Path("idUser") Long idUsuario, @Body PesoDTO nuevoPeso);

}
```

Ilustración 10 - Service de los Usuarios

Por último, se ha creado una clase auxiliar denominada Result, cuyo código se puede observar en la Ilustración 11. Esta clase es utilizada para manejar mensajes con los que se pueda ver si la operación ha salido con éxito o existe algún error. En caso en el que se haya producido un error nos permite mostrar un mensaje al usuario para que en todo momento sepa que está pasando.

5.2.2 Capa View

Esta es la capa que interactúa directamente con el usuario. Para desarrollar dicha capa se han utilizado Activities y Fragments, los cuales tienen como objetivo representar y mostrar los datos al usuario y capturar todos los eventos que se produzcan para que el ViewModel pueda gestionarlos.

Para lograr estos objetivos esta capa se divide en dos partes bien diferenciadas. Por un lado, se definen los Layout XML, en ellos se crean las interfaces de la aplicación con las que el usuario va a interactuar.

```

public abstract class Result<T> {
    2 usages
    private Result() {}

    public static final class Success<T> extends Result<T> {
        public final T datos;

        public Success(T datos) { this.datos = datos; }
    }

    public static final class Error<T> extends Result<T> {
        public final String error;

        public Error(String error) { this.error = error; }
    }
}

```

Ilustración 11 - Clase Result

En la Ilustración 12 se puede ver un ejemplo de cómo se vería la interfaz a través de la que los usuarios pueden consultar los datos de su perfil. Estas interfaces por si solas solo muestran los textos, objetos y etiquetas que creamos en el respectivo XML. Para que el usuario pueda interactuar con la interfaz y pueda disfrutar de todas las facilidades que la aplicación ofrece falta la parte programática de esta capa, las Activities y Fragments.

En esta parte programada en lenguaje Java se escribe el código que hace que se pueda interactuar con la interfaz, con sus botones y textos. En estas clases se crean funciones para que todo funcione correctamente y además, se diseñan funciones que comprueban si se produce algún evento y en el caso en el que ocurra llama al ViewModel para que lo gestione.

Por último, se han creado en algunos casos Adapters para mostrar listas de objetos en la aplicación. Con esta implementación se gestionan grandes listas de información de forma dinámica, mostrando la información de la forma más clara posible.



Ilustración 12 - Interfaz del perfil de un usuario

5.2.3 Capa ViewModel

Esta capa gestiona el estado de la interfaz de la aplicación. En ella lo que se hace es mantener que los datos sean correctos asegurando que la lógica de negocio sea correcta y además esté separada de la presentación. Estas clases extienden ViewModel, con lo que conseguimos que si se produce algún cambio de idioma o rotación de pantalla la aplicación tenga que volver a pedir los datos. Por otro lado, para mantener al usuario siempre informado de los cambios que ocurren se usan objetos LiveData que envían mensajes y se actualizan cuando el ViewModel recibe nueva información.

En Ilustración 13 podemos observar la clase ViewModel que gestiona el login de los usuarios. En ella podemos observar como se usan los distintos objetos LiveData en caso de que el login del usuario sea exitoso u ocurra algún error como por ejemplo que los campos de inicio de sesión no se hayan rellenado o su información no sea válida.

```
public class LoginViewModel extends ViewModel {  
  
    2 usages  
    private final AuthenticationRepository autentificationRepository;  
  
    2 usages  
    private final MutableLiveData<String> mensajeError = new MutableLiveData<>();  
    1 usage  
    private final MutableLiveData<UsuarioDTO> inicioSesionExitoso = new MutableLiveData<>();  
  
    no usages  
    public LoginViewModel() { this.autentificationRepository = new AuthenticationRepository(); }  
  
    1 usage  
    public void login(InicioSesionDTO inicioSesionDTO) {  
        if (inicioSesionDTO.getEmail().isEmpty() || inicioSesionDTO.getContraseña().isEmpty()) {  
            mensajeError.setValue("Por favor, rellena todos los campos");  
            return;  
        }  
  
        autentificationRepository.iniciarSesion(inicioSesionDTO).observeForever( Result<UsuarioDTO> result -> {  
            if (result instanceof Result.Success) {  
                inicioSesionExitoso.setValue(((Result.Success<UsuarioDTO>) result).datos);  
            } else if (result instanceof Result.Error) {  
                mensajeError.setValue(((Result.Error<?>) result).error);  
            }  
        });  
    }  
}
```

Ilustración 13 - Clase ViewModel del Login de los Usuarios

6. PRUEBAS

Tras la fase de implementación de la aplicación ahora se debe comprobar que todo funciona correctamente. Para ello se han de realizar distintas pruebas que confirmen que se cumplen correctamente los requisitos funcionales que se definieron en la fase de análisis.

Para cubrir todo el proyecto se han dividido las pruebas en dos grandes grupos. Por un lado, las pruebas del backend del servicio REST y por otro lado pruebas del frontend de la aplicación Android.

6.1 Pruebas del backend

En este apartado vamos a comprobar el correcto funcionamiento de la lógica de negocio del backend de la aplicación, el cual nos proporciona todos los servicios que se usan en el frontend. Esta sección se ha dividido a su vez en dos tipos de pruebas: unitarias y de integración.

6.1.1 Pruebas unitarias

El objetivo de estas pruebas es verificar el funcionamiento de una clase de forma aislada. Para lograrlo se ha utilizado el framework JUnit junto a la librería Mockito para la utilización de objetos Mock.

Este tipo de prueba se ha aplicado únicamente en el caso del servicio `IngredientesService` porque contiene la lógica que se encarga del cálculo de los nutrientes. Para el resto de clases, que únicamente implementan funcionalidades CRUD, se considera suficiente con aplicar pruebas de integración.

A modo de ejemplo, en la Ilustración 14 se muestra un fragmento de esta prueba, donde se observa la declaración y programación de los objetos Mock y las aserciones aplicadas para uno de los casos no válidos del método `crearIngrediente`.

6.1.2 Pruebas de integración

Tras realizar las pruebas unitarias en una clase aislada, se pasa a comprobar el funcionamiento de varias clases en conjunto a través de pruebas de integración.

Por un lado, se han implementado pruebas de integración usando el framework `SpringBootTest`, cargando el contexto de la aplicación y utilizando las implementaciones reales de los servicios, repositorios y la base de datos.

```

public class IngredientesServiceTest {
    @Mock
    private ComidaRepository comidaRepository;
    ...
    @InjectMocks
    private IngredienteService ingredienteService;

    @Test
    void CreaIngredienteGramosNegativosTest() {
        Comida comida = new Comida(1L);
        IngredienteDTO dto = new IngredienteDTO(-50);
        when(comidaRepository.findById(1L)).thenReturn(Optional.of(comida));
        assertThrows(CantidadNegativaException.class,
            () -> ingredienteService.crearIngrediente(1L, dto)
        );
    }
}

```

Ilustración 14 - Ejemplo prueba unitaria con objetos Mock

En concreto, se han aplicado al servicio que se encarga del registro de usuarios y la autenticación, `UsuariosService`, comprobando el proceso de verificación del usuario, la detección de correo duplicado, el cifrado de las contraseñas y la validación de las credenciales. En la Ilustración 15 se puede ver un ejemplo de una de estas pruebas, donde se comprueba que el usuario ha sido debidamente registrado y su contraseña encriptada correctamente. Se puede observar cómo se usa inyección de dependencias (anotación `@Autowired`) para obtener las instancias de los componentes implicados en la prueba.

```

@SpringBootTest
@Transactional
public class UsuarioServiceIntegracionTest {
    @Autowired
    private UsuarioService usuarioService;
    @Autowired
    private UsuarioRepository usuarioRepository;
    @Autowired
    private PasswordEncoder passwordEncoder;

    @Test
    void registraUsuarioValidoTest() {
        RegistroDTO dto = crearRegistroDTO();
        UsuarioDTO resultado = usuarioService.registrarUsuario(dto);
        Usuario usuario = usuarioRepository.findById(resultado.getIdUsuario())
            .orElse(null);
        assertEquals("Alex", usuario.getNombre());
        ...
        assertTrue(passwordEncoder.matches("1234",
            usuario.getCredenciales().getContraseña()));
    }
}

```

Ilustración 15 - Ejemplo prueba de integración

Por otro lado, se han implementado pruebas de integración globales, utilizando Postman para comprobar que cada uno de los endpoints del servicio funciona correctamente y cumple con su función en la aplicación. De este modo se comprueba la integración de todos los componentes del backend en conjunto. A su vez fue muy útil para ver que todo el manejo de errores y excepciones era correcto. A continuación, en la Ilustración 16 se muestra un ejemplo de endpoint probado usando Postman, con el cuerpo enviado y la respuesta retornada.

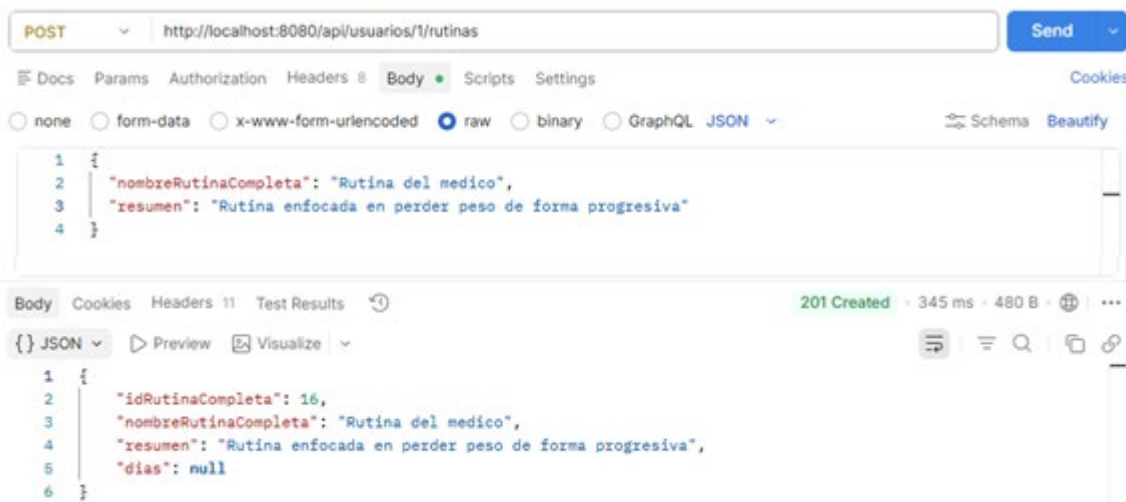


Ilustración 16 - Ejemplo prueba de rutinas en Postman

6.2 Pruebas del frontend

Tras comprobar el correcto funcionamiento del backend y ver que el servicio REST cumple su función, queda por verificar la última parte del mecanismo, el frontend. En esta parte vamos a probar que la interfaz gráfica de las distintas pantallas, la comunicación entre ellas y el acceso a los servicios de la API REST funcionan como deben.

A lo largo de todo el proceso se han probado todas las historias de usuario implementadas. Se han escogido tres casos específicos que se muestran en la Tabla 18 a modo de ejemplo para explicar en la memoria.

Tabla 18 - Pruebas del frontend

ID	Pantalla	Acción	Resultado Esperado
PF-01	Login	Inicio Sesión	El usuario accede a la app
PF-02	Rutinas del usuario	Crear una rutina	La nueva rutina aparece en las listas del usuario
PF-03	Ingredientes de una comida	Añadir un nuevo ingrediente a la comida	Se agrega el ingrediente y se suman sus nutrientes

Para el inicio de sesión se ha tomado un usuario que se registró anteriormente en la aplicación con todos sus datos. A continuación, la Ilustración 17 muestra cómo se vería todo el proceso: el usuario introduciría sus credenciales y en caso de que sean correctas accedería a su pantalla de bienvenida, donde se mostrarían las calorías que debe consumir al día para alcanzar sus objetivos.

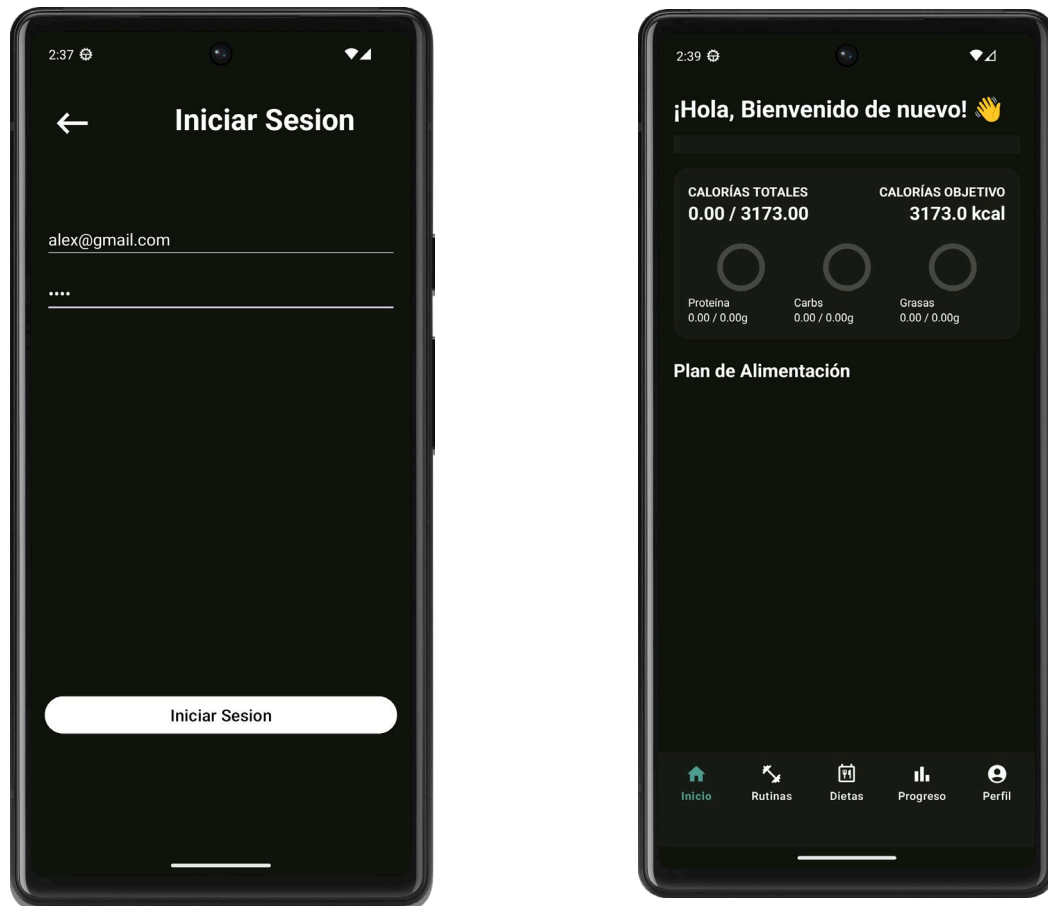


Ilustración 17 - Pantallas de inicio de sesión y de bienvenida

A continuación, la Ilustración 18 muestra el proceso para que un usuario cree su propia rutina de gimnasio. Primero rellenaría los campos de la rutina y al pulsar el botón de crear la rutina se mostraría en la lista con su nombre y descripción.

Por último, la Ilustración 19 muestra cómo se añadiría un ingrediente a una comida de la dieta de un usuario. En la primera imagen se ve como el usuario hace uso del buscador para seleccionar el alimento que quiere agregar a la comida. Después selecciona la cantidad en gramos y el sistema le muestra los valores nutricionales que proporciona ese alimento. Al pulsar el botón de agregar vemos en la pantalla de las comidas como al pulsar la comida se muestran los ingredientes que tiene y como los valores nutricionales se suman al total.

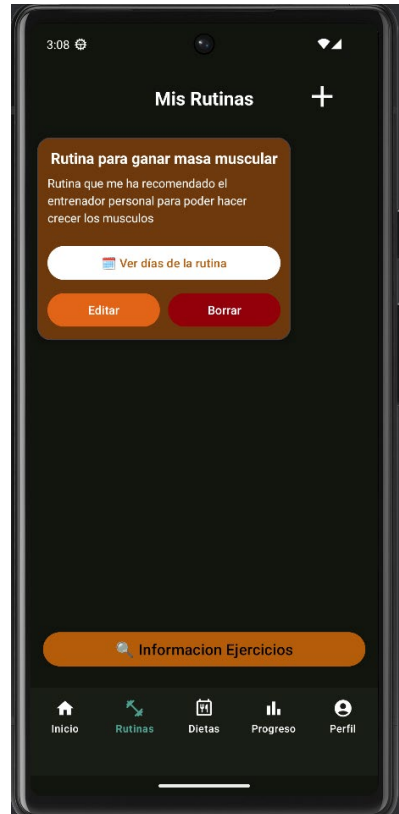
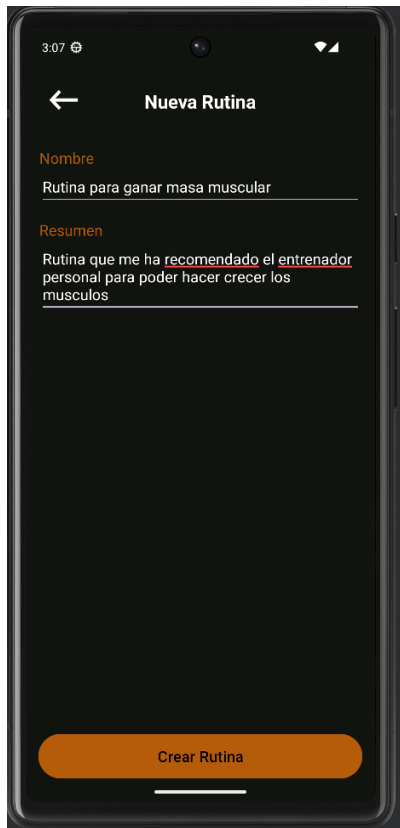


Ilustración 18 - Pantallas para la creación y consulta de rutinas de gimnasio

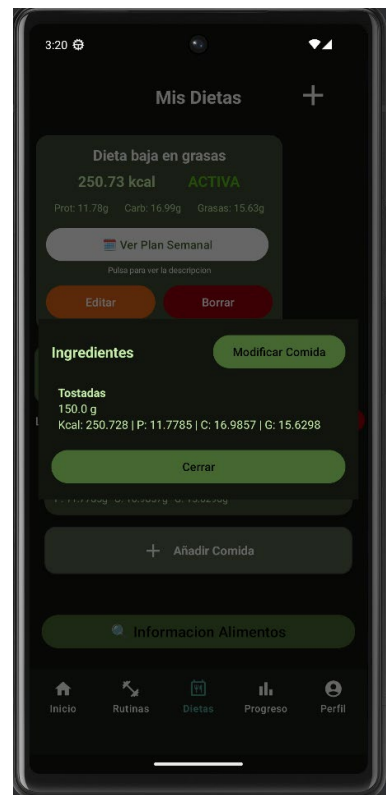


Ilustración 19 - Pantallas para añadir y consultar ingredientes

7. CONCLUSIONES Y TRABAJOS FUTUROS

En este último capítulo tras finalizar el desarrollo de la aplicación se va a valorar si se han alcanzado los objetivos que se habían acordado al inicio del proyecto. Además, se indicarán posibles mejoras y nuevas funcionalidades que se podrían realizar en un futuro de manera que la aplicación siga en constante evolución y continúe desarrollándose.

7.1 Conclusiones

Después de acabar con las pruebas y dar por concluido todo el desarrollo se puede afirmar que se ha alcanzado el objetivo planteado al inicio del Trabajo Fin de Grado. Se ha conseguido desarrollar una aplicación fitness que cuenta tanto con un lugar donde crear tus propias dietas y anotar los valores nutricionales que consumes al día, como un lugar para apuntar todas tus rutinas de gimnasio y tener un conteo del progreso.

Gracias a este proyecto se ha conocido todo el proceso que se sigue a la hora de desarrollar una aplicación móvil desde el comienzo, partiendo de la toma de requisitos hasta el desarrollo de tanto el backend con sus servicios como del frontend y el diseño de la interfaz.

Para todo ello se han utilizado varias tecnologías distintas como Spring Boot, Retrofit, MySQL, Android Studio y varios patrones con los que se han puesto en práctica todos los conocimientos que se han ido adquiriendo a lo largo de toda la carrera en un ambiente más serio y profesional.

7.2 Trabajos Futuros

Por último, aunque la aplicación cumpla con su función y esté acabada el software siempre está en constante cambio y evolución, por lo que en este último apartado se muestra una lista con varias mejoras y nuevas funcionalidades que podrían mejorar la aplicación y la experiencia del usuario.

Algunas de estas mejoras son:

- Implementar una nueva función donde el usuario pueda iniciar una rutina en la que tenga un contador del tiempo que lleva de entrenamiento y le cuente la cantidad de calorías que ha consumido dependiendo del ejercicio que ha hecho.
- Para mejorar la seguridad sería conveniente utilizar JWT para autenticar a los usuarios a la hora de acceder a los servicios.
- Además del gráfico de peso que ya existe en la aplicación estaría bien tener un gráfico de las calorías que ha consumido el usuario a lo largo

de los días para llevar un conteo de cómo lleva su progreso de los valores nutricionales.

- En caso de que un usuario no sepa bien que dieta crear o cual podría ser adecuada para alcanzar sus objetivos estaría bien que la aplicación mostrase ejemplos de dietas en función de los datos que el usuario aportó a la hora de registrarse en la aplicación.
- Permitir al usuario configurar sus propias notificaciones personalizadas ya que ahora mismo las notificaciones que hay son programadas por la aplicación a modo de aviso para recordarle al usuario que tiene que apuntar la comida que ha consumido.
- Desarrollar la aplicación para que sea compatible con otros sistemas operativos como iOS.
- A parte de las fotos de los ejercicios de gimnasio que hay habría que agregar videos didácticos para que el usuario pueda comprender mejor como debe hacer la técnica del ejercicio en cuestión y que no se lesione.

Bibliografía

- [1] Spring, “Spring Initializr”. Disponible en: <https://start.spring.io/>
- [2] Microsoft, “Arquitectura MVVM en .NET MAUI”. Disponible en: <https://learn.microsoft.com/es-es/dotnet/architecture/maui/mvvm>
- [3] PlantUML, “PlantUML Documentation”. Disponible en: <https://plantuml.com/es/>
- [4] Scrumdesk “Agile Project Management Tool”. Disponible en: <https://www.scrumdesk.com/>
- [5] GitHub, “GitHub Dashboard”. Disponible en: <https://github.com/dashboard>
- [6] Retrofit, “Type-safe HTTP client for Android and Java”. Disponible en: <https://square.github.io/retrofit/>
- [7] MySQL, “MySQL” Disponible en: <https://www.mysql.com/>
- [8] Postman, “API Platform for Developers”. Disponible en: <https://www.postman.com/>
- [9] Android Developers, “ Android Developers”. Disponible en: <https://developer.android.com/?hl=es-419>
- [10] Red Hat, “¿Qué es una API REST?” Disponible en: <https://www.redhat.com/es/topics/api/what-is-a-rest-api>
- [11] Microsoft Learn “Implementación del repositorio y los patrones de unidad de trabajo en una aplicación MVC de ASP.NET” Disponible en: <https://learn.microsoft.com/es-es/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application>